

# The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch19\_nlp

AYuSh

# Contents

|                                         |   |
|-----------------------------------------|---|
| Chapter 19: Classical & Neural NLP..... | 3 |
|-----------------------------------------|---|

# Chapter 19: Classical & Neural NLP

---

Part IV begins where language actually gets processed. Everything after this chapter — pretrained LMs, alignment, RAG, agents — assumes the vocabulary built here: what a token is and why the choice matters, how text becomes vectors (sparse counts, then dense embeddings), how sequences get labeled, and where the classical methods break in ways that motivated the neural ones. Interviewers use this material two ways: as fundamentals ("implement TF-IDF", "derive negative sampling") and as judgment probes ("when would TF-IDF plus logistic regression beat a transformer?" — more often than candidates expect: small data, latency budgets, interpretability requirements, and strong lexical signal).

All of it runs on a real corpus (20 Newsgroups): Zipf's law measured at slope  $-0.97$  with 42% of the vocabulary appearing exactly once (Listing 1); TF-IDF built from scratch, worth +2.4 points over raw counts, with the classifier's learned weights recovering the topic lexicons (Listing 2); n-gram language models where add-one smoothing is measured making a trigram model *nine times worse* than a unigram baseline — and interpolation fixing it (Listing 3); skip-gram with negative sampling trained from scratch until *nasa* neighbors *ames, jsc, astronaut* and  $\cos(\text{shuttle}, \text{goalie}) = -0.80$ , including the anisotropy artifact that makes raw embedding cosines lie (Listing 4); BPE learned from data, tokenizing never-seen words into sensible morphemes (Listing 5); an HMM tagger whose Viterbi decoding is verified equal to brute-force search over 3,125 paths, then beats greedy 0.956 to 0.807 on ambiguous words (Listing 6); a structured perceptron doing BIO-scheme NER at span-F1 0.936 (Listing 7); and the negation problem measured: unigram sentiment 0.619, bigrams or negation-marking 1.000 (Listing 8).

## Preprocessing and the shape of text

Text arrives as strings; every pipeline decision that turns them into units is a modeling decision wearing a plumbing costume. **Tokenization** (Listing 1 uses the crudest: lowercase alphabetic spans) fixes what can ever be learned — split badly and no downstream model recovers. **Stopword removal** deletes high-frequency function words: 19 words account for 24% of the corpus; classical retrieval loved the compression, but "to be or not to be" dissolves entirely, and modern models keep stopwords because syntax lives there. **Stemming** (crude suffix rules; Porter's algorithm is the canonical version) and **lemmatization** (dictionary-based, POS-aware: *better* → *good*) merge inflections to fight sparsity — Listing 1's ten-rule stemmer cuts the vocabulary 17% and shows the price: *news* → *new* (false merge), *flies* → *fli* (non-word), *university* untouched (missed merge). Sub-word tokenization (below) made most of this obsolete by making the units learnable.

The statistical fact underneath: **Zipf's law**, frequency  $\propto 1/\text{rank}$  — measured on the corpus at log-log slope  $-0.97$ . Consequences worth reciting: the top 100 word types cover 42% of all tokens while 42% of the vocabulary occurs exactly *once* (hapax legomena), so any fixed vocabulary wastes most of its slots on words with almost no training signal — the argument that ends in BPE; and n-gram counts go sparse exponentially fast — the argument that ends in smoothing.

## Bag-of-words and TF-IDF

The bag-of-words representation throws away order and keeps counts: a document is a vector over the vocabulary. Raw counts have two defects — long documents dominate, and ubiquitous words swamp informative ones. **TF-IDF** fixes both: weight term frequency by  $\log(N/\text{df})$  (rarity across documents), then L2-normalize rows. Listing 2 builds it from scratch on space-vs-hockey classification: logistic regression scores 0.919 on raw counts, 0.943 on TF-IDF, and the learned weights *are* the topic lexicons (*nasa, launch, orbit, moon* vs *hockey, team, season, players* — the figure). Details interviews probe: the vocabulary comes from *train only* (test-time unknown words are dropped — the OOV problem, honestly present in the listing); df smoothing (+1) avoids division by zero; and the whole pipeline — sparse features, linear model — remains the strongest baseline per unit of compute in NLP. **n-grams** as features restore local order at exponential vocabulary cost; Listing 8 shows the one case they're indispensable.

## n-gram language models

Before neural LMs,  $p(w_t | w_{t-n+1..t-1})$  was a lookup table of counts, and the entire craft was handling zeros: any unseen trigram makes a sentence probability zero and perplexity infinite. Listing 3 measures the classic solutions on real text with proper machinery (train/test split, `<unk>` for rare words, sentence padding). The star result is a negative one: **add-one (Laplace) smoothing makes the trigram model catastrophically worse than a unigram baseline** — perplexity 3,891 vs 438 — because adding 1 to all  $V = 6,350$  unseen continuations steals nearly all probability mass from the few dozen actually-observed ones; shrinking  $k$  to 0.01 only halves the damage. **Interpolation** (mix trigram, bigram, unigram estimates) drops perplexity to 290 — beating everything — because backing off to shorter contexts is statistically honest about sparsity. The lineage to name: Katz backoff, and **Kneser-Ney** (the classical state of the art), whose insight — continuation probability: *how many contexts* a word completes, not how often it appears — deserves a sentence in any answer. The generated samples show n-gram fluency's ceiling: locally plausible, globally incoherent — no memory beyond two words. **Perplexity** ( $\exp$  of mean negative log-likelihood; the effective branching factor) recurs in Chapter 20 as the pretraining metric, with the same caveat it has here: comparable only at equal vocabulary and tokenization.

## Word embeddings

The bag-of-words has no notion that *good* and *great* are related — every word is orthogonal. **Word2vec** (Mikolov et al., 2013) learns dense vectors from the distributional signal: words in similar contexts get similar vectors. Two architectures — **CBOW** (predict center from context average; faster, better for frequent words) and **skip-gram** (predict each context word from center; better for rare words) — and one crucial trick: the honest objective is a softmax over the full vocabulary, unaffordable at every step, so **negative sampling** reframes it as logistic discrimination — push the dot product up for the true (center, context) pair, down for  $k$  sampled fake contexts (drawn from the unigram distribution raised to  $3/4$ , which up-weights mid-frequency words). Listing 4 implements the whole recipe from scratch — frequent-word subsampling ( $t = 10^{-4}$ ), the  $u^{3/4}$  noise table, both embedding matrices — and earns its results on a two-topic corpus: *nasa* → *ames, jsc, astronaut, lunar*; *hockey* → *nhl, championship, league*;  $\cos(\text{shuttle}, \text{goalie}) = -0.80$  across topics,  $+0.82$  for (*shuttle, launch*) within.

The listing preserves two debugging lessons. First, an undertrained run (one context per center, few epochs) produced embeddings where *every* cosine exceeded 0.99 — collapsed, not converged; the fix was training on all  $2W$  window contexts with a decayed learning rate. Second, even the good run shows **anisotropy** — raw mean pairwise cosine 0.59, because all vectors share a common direction; subtracting the mean vector before measuring (mean cosine → 0.10) is standard hygiene that most tutorials omit and real papers (all-but-the-top) formalize. Complete the family for interviews: **GloVe** fits log co-occurrence counts globally (matrix factorization view — and SGNS itself implicitly factorizes a shifted PMI matrix, the Levy-Goldberg result); **FastText** builds word vectors from character n-grams, buying OOV vectors and morphology; and all of them are *static* — one vector per type, so *bank* (river) and *bank* (money) collide — the deficiency contextual embeddings (Chapter 20) exist to fix.

## Subword tokenization

Fixed word vocabularies fail Zipf: unbounded types, most rare, OOV guaranteed. **BPE** (Sennrich et al., 2016) solves it bottom-up: start from characters, repeatedly merge the most frequent adjacent pair, stop at a target vocabulary size. Listing 5 learns 2,000 merges from the corpus and shows the machine working: the first merges are English's skeleton (*e+*, *th+e*); average tokens-per-word falls  $5.5 \rightarrow 1.6$  (the figure's curve *is* the vocabulary-size/sequence-length trade every LLM tokenizer picks a point on); frequent words become single tokens while never-seen words decompose sensibly — *astrophysicist* → *astro* + *physici* + *st*, *reusability* → *re* + *us* + *ability* — and worst-case gibberish (zzyzx) falls back to characters. Nothing is ever OOV again. The family tree to recite: **WordPiece** (BERT) merges by likelihood gain rather than frequency; **Unigram LM** (used via SentencePiece) goes top-down — start big, prune tokens by likelihood loss, enabling probabilistic segmentation and subword regularization; **SentencePiece** operates on raw text including spaces (no pre-tokenization, language-agnostic); byte-level BPE (GPT-2+)

starts from 256 bytes so even emoji decompose. Interview-current footnote: tokenization is why LLMs are bad at character-level tasks (counting letters in *strawberry* — the model literally does not see letters), why arithmetic is token-fragile, and why low-resource languages pay a multiple in tokens-per-sentence.

## Sequence labeling: POS and NER

Tagging tasks assign a label per token, and the interview kernel is the same everywhere: *decisions interact*, so decode jointly. Listing 6 builds the classical machine — an HMM: transition matrix  $A$  (tag  $\rightarrow$  tag), emission matrix  $B$  (tag  $\rightarrow$  word), estimated by smoothed counting — and **Viterbi**, the dynamic program that finds the exact argmax path in  $O(nT^2)$  instead of  $T^n$ ; the listing *verifies exactness* against brute-force enumeration of all  $5^5$  paths. On data with genuine lexical ambiguity (*light, flies, back, fast* each valid under multiple tags), the ordering is the lesson: most-frequent-tag baseline 0.746, greedy left-to-right 0.807, Viterbi 0.956 — and the garden-path sentence "the light flies quickly" gets greedy-tagged as ADJ NOUN (then stuck) but Viterbi-tagged DET NOUN VERB ADV, because joint decoding lets later words veto earlier choices.

**NER** upgrades the label scheme to spans via **BIO** (Begin/Inside/Outside per entity type). Listing 7 trains a **structured perceptron** — score = feature weights + transition weights, Viterbi decode, update on the *difference* between gold and predicted paths (Collins 2002, the simplest discriminative sequence model and the direct ancestor of CRF training) — with word/context features, reaching **span-level F1 0.936**, the metric worth explaining: a half-found entity scores zero, so span F1 is deliberately brutal, and it's the standard. The classical-to-neural bridge to narrate: HMM (generative, count-based)  $\rightarrow$  CRF (discriminative, feature-based, globally normalized)  $\rightarrow$  BiLSTM-CRF (learned features, Chapter 15)  $\rightarrow$  fine-tuned transformers (Chapter 20), each step replacing hand-built structure with learned structure. Dependency parsing completes the classical syllabus — transition-based (shift/reduce, greedy, fast) vs graph-based (maximum spanning tree, exact) — one sentence each is what interviews want.

## Text classification and the negation problem

Listing 8 closes with the failure that separates representations. Sentiment data where 40% of polarity words are *negated* ("not good" in a positive... no — meaning follows the negation): unigram bag-of-words scores **0.619** — it sees *good* and votes positive, blind to the *not* sitting adjacent, and the errors are precisely the negated sentences. Two classical repairs, both measured at 1.000: **bigrams** (the pair "not good" becomes its own feature) and **negation marking** (the pre-neural standard: rewrite tokens after a negator as *good\_NEG*, flipping their identity). The general lesson outlives sentiment: bag-of-words fails wherever meaning is *compositional* — negation, sarcasm, scope, comparatives ("better than nothing") — and each repair is a hand-built approximation of what attention (Chapter 16) learns wholesale. Said as deployment judgment: TF-IDF + linear on lexical

tasks (topic, spam, routing) remains fast, interpretable, and strong; the moment labels hinge on composition, order, or context, the classical ceiling appears exactly where Listing 8 put it.

## Code implementations

### Listing 1 — Preprocessing and Zipf's law: what tokenization decisions cost and buy

```

"""Listing 1: preprocessing and Zipf's law -- what tokenization decisions cost and buy."""
import re, numpy as np
from collections import Counter
from sklearn.datasets import fetch_20newsgroups
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

data = fetch_20newsgroups(subset='train', categories=['sci.space', 'rec.sport.hockey'],
                        remove=('headers', 'footers', 'quotes'))
text = "\n".join(data.data)
print(f"corpus: {len(data.data)} documents, {len(text):,} characters")

tokens = re.findall(r"[a-z]+", text.lower())
counts = Counter(tokens)
print(f"tokens {len(tokens):,} vocabulary {len(counts):,}")
print(f"hapax legomena (words seen ONCE): {sum(1 for c in counts.values() if c==1):,} "
      f"({100*sum(1 for c in counts.values() if c==1)/len(counts):.0f}% of vocab)")

top = counts.most_common(10)
print("top-10:", [w for w, _ in top])
stop = set("the a an of to and in is that it for on was as with be this are
at".split())
kept = [t for t in tokens if t not in stop]
print(f"after a 19-word stopwords list: {len(kept):,} tokens ({100*(1-len(kept)/
len(tokens)):.0f}% of the corpus was stopwords)")

# crude suffix-stripping stemmer vs what it breaks
def stem(w):
    for s in ("ational", "tional", "iveness", "fulness", "ing", "ed", "es", "s", "ly", "er"):
        if w.endswith(s) and len(w) > len(s)+2: return w[:len(w)-len(s)]
    return w
merged = Counter(stem(w) for w in counts)
print(f"crude stemming: vocab {len(counts):,} -> {len(merged):,}")
print("damage: news ->", stem("news"), "(merges with 'new'); flies ->", stem("flies"),
      "(a non-word); university ->", stem("university"), "(untouched: rules miss it)")

# Zipf: frequency ~ 1/rank
freqs = np.array(sorted(counts.values(), reverse=True), float)
ranks = np.arange(1, len(freqs)+1)
sl = np.polyfit(np.log(ranks[:3000]), np.log(freqs[:3000]), 1)[0]
print(f"Zipf log-log slope over top 3000 ranks: {sl:.2f} (Zipf's law predicts ~ -1)")
cover = np.cumsum(freqs)/freqs.sum()
for k in [100, 1000, 10000]:
    print(f"top {k:5d} word types cover {100*cover[k-1]:.1f}% of all tokens")

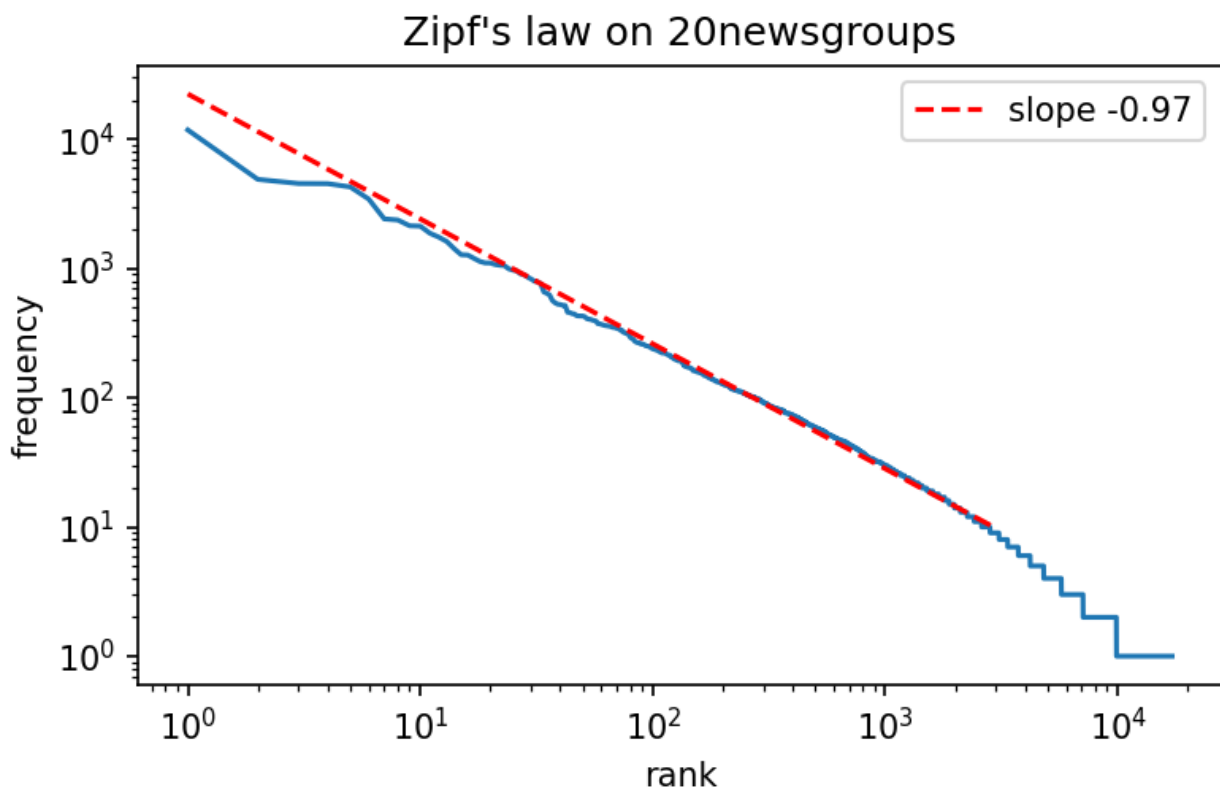
fig, ax = plt.subplots(figsize=(5.4, 3.6))
ax.loglog(ranks, freqs); ax.loglog(ranks[:3000], np.exp(np.polyval(np.polyfit(
    np.log(ranks[:3000]), np.log(freqs[:3000]), 1), np.log(ranks[:3000]))), 'r--',
label=f'slope {sl:.2f}')
ax.set(xlabel='rank', ylabel='frequency', title="Zipf's law on 20newsgroups");
ax.legend()

```

```
fig.tight_layout(); fig.savefig("figures/ch19/fig_l1_zipf.png", dpi=150)
print("figure saved: fig_l1_zipf.png")
```

Output:

```
corpus: 1193 documents, 1,513,176 characters
tokens 219,950   vocabulary 17,135
hapax legomena (words seen ONCE): 7,272 (42% of vocab)
top-10: ['the', 'to', 'a', 'of', 'and', 'in', 'i', 'is', 'for', 'that']
after a 19-word stopwords list: 167,171 tokens (24% of the corpus was stopwords)
crude stemming: vocab 17,135 -> 14,153
damage: news -> new (merges with 'new'); flies -> fli (a non-word); university ->
university (untouched: rules miss it)
Zipf log-log slope over top 3000 ranks: -0.97 (Zipf's law predicts ~ -1)
top 100 word types cover 42.2% of all tokens
top 1000 word types cover 71.1% of all tokens
top 10000 word types cover 96.8% of all tokens
figure saved: fig_l1_zipf.png
```



## Listing 2 — Bag-of-words and TF-IDF from scratch -- and why raw counts mislead

```
"""Listing 2: bag-of-words and TF-IDF from scratch -- and why raw counts mislead."""
import re, numpy as np
from collections import Counter
from sklearn.datasets import fetch_20newsgroups
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
```



```

cats = ['sci.space', 'rec.sport.hockey']
tr = fetch_20newsgroups(subset='train', categories=cats,
remove=('headers', 'footers', 'quotes'))
te = fetch_20newsgroups(subset='test', categories=cats,
remove=('headers', 'footers', 'quotes'))
tok = lambda t: re.findall(r"[a-z]+", t.lower())

# vocabulary from train only (test words unseen in train are DROPPED -- the OOV
reality)
vocab = {}
for t in tr.data:
    for w in tok(t):
        if w not in vocab and len(vocab) < 20000: vocab[w] = len(vocab)
V = len(vocab)

def bow(docs):
    Xm = np.zeros((len(docs), V))
    for i, t in enumerate(docs):
        for w in tok(t):
            if w in vocab: Xm[i, vocab[w]] += 1
    return Xm
Xtr, Xte = bow(tr.data), bow(te.data)
ytr, yte = tr.target, te.target

# TF-IDF: tf x log(N/df), then L2-normalize rows
df = (Xtr > 0).sum(0)
idf = np.log((1+len(Xtr))/(1+df)) + 1
def tfidf(Xm):
    W = Xm*idf
    return W/(np.linalg.norm(W, axis=1, keepdims=True)+1e-12)
Ttr, Tte = tfidf(Xtr), tfidf(Xte)

# highest-weight words for one doc under counts vs tf-idf
i = 5
top_raw = np.argsort(Xtr[i][:,-1][:6])
top_tfidf = np.argsort(Ttr[i][:,-1][:6])
inv = {v: k for k, v in vocab.items()}
print("doc 5, top words by RAW COUNT:", [inv[j] for j in top_raw])
print("doc 5, top words by TF-IDF: ", [inv[j] for j in top_tfidf])

def train_lr(Xm, y, iters=300, lr=0.5):
    w = np.zeros(Xm.shape[1]); b = 0.0
    for _ in range(iters):
        p = 1/(1+np.exp(-np.clip(Xm@w+b, -30, 30)))
        g = Xm.T@(p-y)/len(y); w -= lr*g; b -= lr*(p-y).mean()
    return w, b
for name, A, B in [("raw counts", Xtr, Xte), ("TF-IDF", Ttr, Tte)]:
    w, b = train_lr(A, ytr)
    acc = ((B@w+b) > 0) == yte).mean()
    print(f"logistic regression on {name:10s}: test acc {acc:.3f}")

w, b = train_lr(Ttr, ytr)
top_pos = np.argsort(w)[:,-1][:8]; top_neg = np.argsort(w)[:8]
print(f"most {tr.target_names[1]}-ish words:", [inv[j] for j in top_pos])
print(f"most {tr.target_names[0]}-ish words:", [inv[j] for j in top_neg])

fig, ax = plt.subplots(figsize=(6.4, 3.4))
srt = np.argsort(w)
sel = np.concatenate([srt[:8], srt[-8:]])
ax.barh([inv[j] for j in sel], w[sel], color=['tab:blue']*8+['tab:orange']*8)
ax.set(title='LR weights on TF-IDF (blue=hockey side, orange=space side)')
fig.tight_layout(); fig.savefig("figures/ch19/fig_l2_tfidf.png", dpi=150)
print("figure saved: fig_l2_tfidf.png")

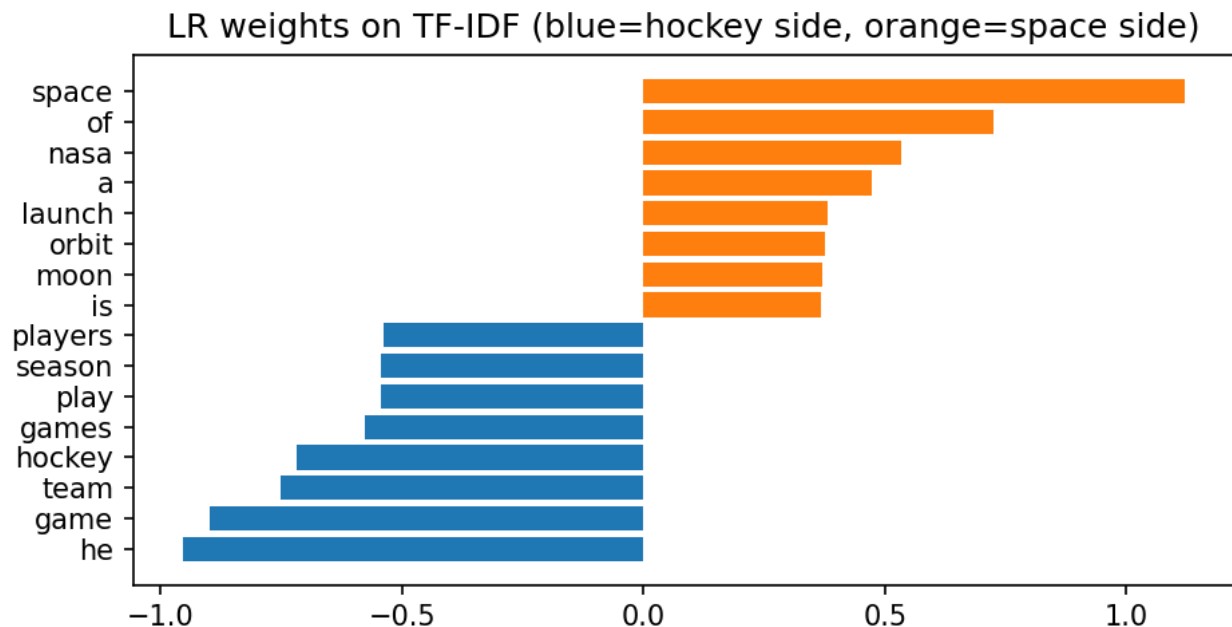
```

Output:

```

doc 5, top words by RAW COUNT: ['i', 'think', 'all', 'cobra', 'expected', 'then']
doc 5, top words by TF-IDF:    ['think', 'different', 'i', 'voting', 'yup', 'cobra']
logistic regression on raw counts: test acc 0.919
logistic regression on TF-IDF   : test acc 0.943
most sci.space-ish words: ['space', 'of', 'nasa', 'a', 'launch', 'orbit', 'moon', 'is']
most rec.sport.hockey-ish words: ['he', 'game', 'team', 'hockey', 'games', 'play',
'season', 'players']
figure saved: fig_l2_tfidf.png

```



### Listing 3 — n-gram language models: smoothing, backoff, and perplexity

```

"""Listing 3: n-gram language models -- smoothing, backoff, and perplexity."""
import re, numpy as np
from collections import Counter, defaultdict
from sklearn.datasets import fetch_20newsgroups
rng = np.random.default_rng(3)

data = fetch_20newsgroups(subset='train', categories=['sci.space'],
remove=('headers', 'footers', 'quotes'))
docs = [re.findall(r"[a-z']+|[\.\!?\"]", d.lower()) for d in data.data]
docs = [d for d in docs if len(d) > 10]
cut = int(len(docs)*0.9)
train, test = docs[:cut], docs[cut:]
tokens = [w for d in train for w in d]
vocab = {w for w, c in Counter(tokens).items() if c >= 2} | {"<unk>", "<s>", "</s>"}
V = len(vocab)
prep = lambda d: ["<s>", "<s>"] + [w if w in vocab else "<unk>" for w in d] + ["</s>"]
print(f"train sentences {len(train)}   vocab {V:,}")

uni = Counter(); bi = Counter(); tri = Counter(); bctx = Counter(); tctx = Counter()
for d in map(prepare, train):
    for i in range(2, len(d)):
        uni[d[i]] += 1
        bi[(d[i-1], d[i])] += 1; bctx[d[i-1]] += 1
        tri[(d[i-2], d[i-1], d[i])] += 1; tctx[(d[i-2], d[i-1])] += 1
N = sum(uni.values())

```

```

def p_unigram(w): return uni[w]/N
def p_laplace_tri(a, b, w, k=1.0):
    return (tri[(a,b,w)] + k)/(tctx[(a,b)] + k*V)
def p_interp(a, b, w, l3=.5, l2=.3, l1=.2):
    p3 = tri[(a,b,w)]/tctx[(a,b)] if tctx[(a,b)] else 0
    p2 = bi[(b,w)]/bctx[b] if bctx[b] else 0
    return l3*p3 + l2*p2 + l1*p_unigram(w)

def perplexity(pfun):
    ll, n = 0, 0
    for d in map(prepare, test):
        for i in range(2, len(d)):
            ll += np.log(pfun(d[i-2], d[i-1], d[i]) + 1e-12); n += 1
    return np.exp(-ll/n)

print(f"perplexity, unigram baseline: {perplexity(lambda a,b,w:
p_unigram(w)):8.1f}")
print(f"perplexity, trigram + Laplace k=1: {perplexity(p_laplace_tri):8.1f}
(V={V:,} smothers the counts)")
print(f"perplexity, trigram + Laplace k=0.01: {perplexity(lambda a,b,w:
p_laplace_tri(a,b,w,.01)):8.1f}")
print(f"perplexity, interpolated tri+bi+uni: {perplexity(p_interp):8.1f}")

# generation: sample from the interpolated model
def gen(maxlen=25):
    a, b, out = "<s>", "<s>", []
    words = list(vocab)
    for _ in range(maxlen):
        ps = np.array([p_interp(a, b, w) for w in words]); ps /= ps.sum()
        w = words[rng.choice(len(words), p=ps)]
        if w == "</s>": break
        out.append(w); a, b = b, w
    return " ".join(out)
for i in range(3): print(f"sample {i+1}: {gen()}")

```

Output:

```

train sentences 511   vocab 6,350
perplexity, unigram baseline:          438.0
perplexity, trigram + Laplace k=1:    3890.6   (V=6,350 smothers the counts)
perplexity, trigram + Laplace k=0.01: 1653.2
perplexity, interpolated tri+bi+uni:   289.6
sample 1: from the space symposium on electromagnetic launcher . timing would have each
of gates you find .
sample 2: i have forgotten that . . space foundation use so from lighter i'm sure but
mostly already bright and even our detectors play but i
sample 3: archive name space controversy last faq . a thu apr nasa langley technical
reports are due to the mission control system <unk> . c .

```

## Listing 4 — word2vec skip-gram with negative sampling, from scratch

```

"""Listing 4: word2vec skip-gram with negative sampling, from scratch."""
import re, numpy as np
from collections import Counter
from sklearn.datasets import fetch_20newsgroups
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)

data = fetch_20newsgroups(subset='train', categories=['sci.space', 'rec.sport.hockey'],
                        remove=(['headers', 'footers', 'quotes']))
tokens = re.findall(r"[a-z]+", "\n".join(data.data).lower())
counts = Counter(tokens)
vocab = {w: i for i, (w, c) in enumerate(counts.most_common()) if c >= 5}

```

```

V = len(vocab); ids = np.array([vocab[w] for w in tokens if w in vocab])
print(f"corpus {len(ids):,} tokens   vocab {V:,}")

# subsample frequent words (word2vec's t=1e-4 trick): keeps 'orbit', drops most 'the'
freq = np.bincount(ids, minlength=V)/len(ids)
keep_p = np.minimum(1, np.sqrt(1e-4/freq[ids]))
ids = ids[rng.random(len(ids)) < keep_p]
print(f"after frequent-word subsampling: {len(ids):,} tokens")

D, W, NEG = 50, 4, 5
Wi = (rng.random((V, D))-0.5)/D; Wo = np.zeros((V, D))           # input and output
embeddings
noise = freq**0.75; noise /= noise.sum()                         # unigram^(3/4)
negative table
sig = lambda z: 1/(1+np.exp(-np.clip(z, -30, 30)))

lr0, total = 0.025, 8
step = 0
for epoch in range(total):
    for s0 in range(0, len(ids)-3*W-512, 512):
        centers = ids[s0+W:s0+W+512]
        pos = np.arange(s0+W, s0+W+len(centers))
        for off in list(range(-W, 0)) + list(range(1, W+1)):      # ALL contexts in the
window
            lr = lr0*max(0.1, 1 - step/(total*160*2*W)); step += 1
            ctx = ids[pos + off]
            neg = rng.choice(V, (len(centers), NEG), p=noise)
            vi = Wi[centers]
            po = sig((vi*Wo[ctx]).sum(1)); gpos = (po - 1)[:, None]
            pn = sig(np.einsum('bd,bnd->bn', vi, Wo[neg])); gneg = pn
            dvi = gpos*Wo[ctx] + np.einsum('bn,bnd->bd', gneg, Wo[neg])
            np.add.at(Wo, ctx, -lr*gpos*vi)
            np.add.at(Wo, neg, -lr*gneg[:, :, None]*vi[:, None, :])
            np.add.at(Wi, centers, -lr*dvi)
        print(f"epoch {epoch+1} done")

# raw embeddings are ANISOTROPIC -- a shared dominant direction inflates all cosines.
Eraw = Wi/np.linalg.norm(Wi, axis=1, keepdims=True)
print(f"mean pairwise cosine, raw: {(Eraw[:500]@Eraw[:500].T).mean():.3f} (a common-
direction artifact)")
Wc = Wi - Wi.mean(0)                                           # remove the common component
E = Wc/np.linalg.norm(Wc, axis=1, keepdims=True)
print(f"mean pairwise cosine, centered: {(E[:500]@E[:500].T).mean():.3f}")
inv = {i: w for w, i in vocab.items()}
def near(w, k=6):
    s = E@E[vocab[w]]
    return [inv[j] for j in np.argsort(s)[::-1][1:k+1]]
for w in ["nasa", "hockey", "orbit", "goal"]:
    print(f"nearest to {w:7s}: {near(w)}")

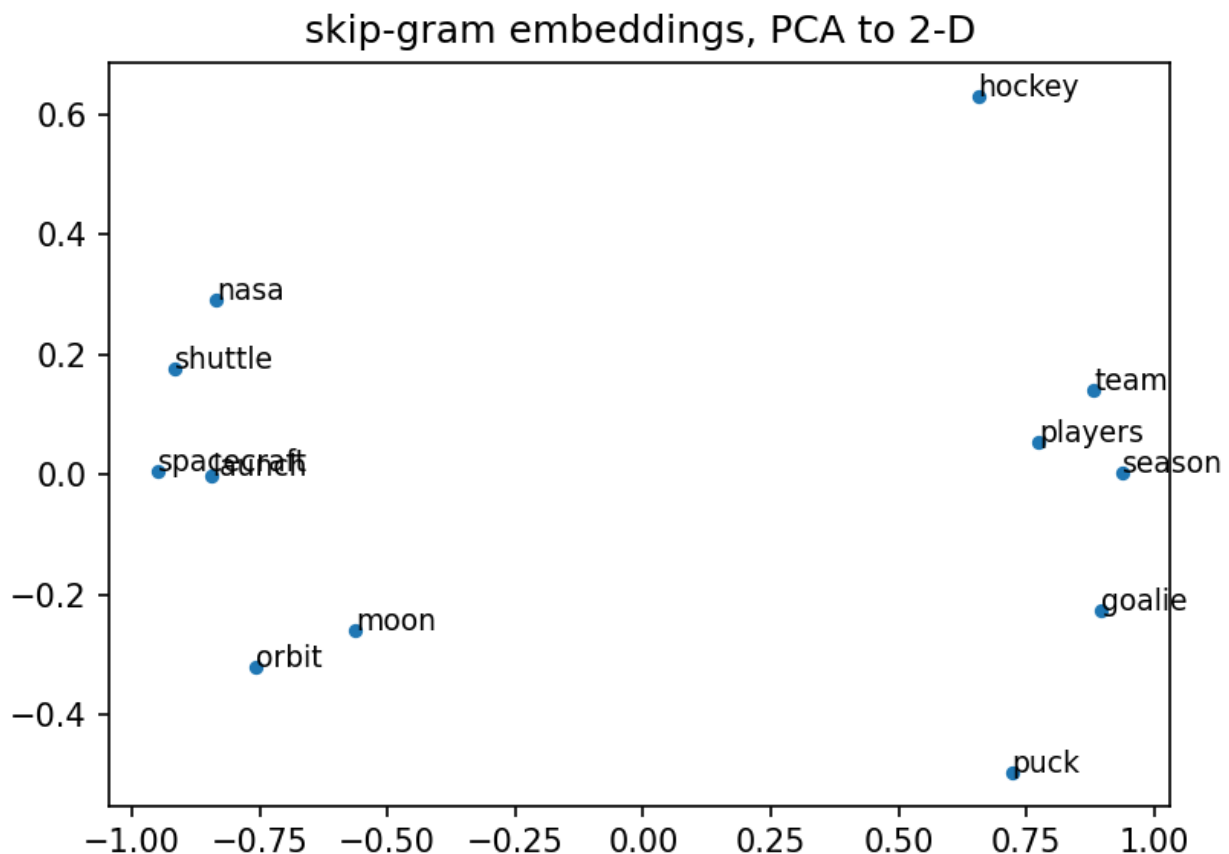
pairs = [("shuttle", "launch"), ("shuttle", "goalie"), ("team", "players"),
("moon", "puck")]
for a, b in pairs:
    print(f"cos({a}, {b}) = {E[vocab[a]]@E[vocab[b]]:.3f}")

words =
["nasa", "orbit", "shuttle", "launch", "moon", "spacecraft", "hockey", "goalie", "puck", "team", "season", "pla
sub = E[[vocab[w] for w in words]]
sub = sub - sub.mean(0)
U, S, Vt = np.linalg.svd(sub, full_matrices=False)
XY = sub@Vt[:2].T
fig, ax = plt.subplots(figsize=(5.6, 4))
ax.scatter(XY[:,0], XY[:,1], s=12)
for (x, yy), w in zip(XY, words): ax.annotate(w, (x, yy), fontsize=9)
ax.set_title('skip-gram embeddings, PCA to 2-D')
fig.tight_layout(); fig.savefig("figures/ch19/fig_l4_w2v.png", dpi=150)
print("figure saved: fig_l4_w2v.png")

```

Output:

```
corpus 199,345 tokens   vocab 4,799
after frequent-word subsampling: 85,875 tokens
epoch 1 done
epoch 2 done
epoch 3 done
epoch 4 done
epoch 5 done
epoch 6 done
epoch 7 done
epoch 8 done
mean pairwise cosine, raw: 0.588 (a common-direction artifact)
mean pairwise cosine, centered: 0.105
nearest to nasa   : ['ames', 'describing', 'monthly', 'astronaut', 'jsc', 'mnemonics']
nearest to hockey : ['championship', 'nhl', 'regulars', 'pool', 'stats', 'league']
nearest to orbit  : ['earth', 'altitude', 'stage', 'impact', 'environment', 'required']
nearest to goal   : ['behind', 'stick', 'soderstrom', 'rebound', 'flyers', 'line']
cos(shuttle, launch) = +0.822
cos(shuttle, goalie) = -0.795
cos(team, players) = +0.861
cos(moon, puck) = -0.304
figure saved: fig_l4_w2v.png
```



### Listing 5 — Byte-pair encoding from scratch: how subwords kill OOV

```
"""Listing 5: byte-pair encoding from scratch -- how subwords kill OOV."""
import re, numpy as np
from collections import Counter
from sklearn.datasets import fetch_20newsgroups
```

```

import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

data = fetch_20newsgroups(subset='train', categories=['sci.space'],
remove=('headers', 'footers', 'quotes'))
words = Counter(re.findall(r"[a-z]+", "\n".join(data.data).lower()))
print(f"training words: {sum(words.values()):,} tokens, {len(words):,} types")

# represent each word as symbols with an end-of-word marker
def to_syms(w): return tuple(w) + ("</w>",)
corpus = {to_syms(w): c for w, c in words.items()}

def merge_step(corpus):
    pairs = Counter()
    for syms, c in corpus.items():
        for i in range(len(syms)-1): pairs[(syms[i], syms[i+1])] += c
    if not pairs: return None, None
    best = max(pairs, key=pairs.get)
    new = {}
    for syms, c in corpus.items():
        out, i = [], 0
        while i < len(syms):
            if i < len(syms)-1 and (syms[i], syms[i+1]) == best:
                out.append(syms[i]+syms[i+1]); i += 2
            else:
                out.append(syms[i]); i += 1
        new[tuple(out)] = c
    return new, best

merges, sizes, lens = [], [], []
for k in range(2000):
    corpus, best = merge_step(corpus)
    if corpus is None: break
    merges.append(best)
    if k % 400 == 0 or k in (9, 99):
        ntok = sum(len(s)*c for s, c in corpus.items()); nw = sum(words.values())
        sizes.append(k+1); lens.append(ntok/nw)
        print(f"after {k+1:4d} merges: avg {ntok/nw:.2f} subwords/word   latest merge:
{best[0]!r}+{best[1]!r}")

def encode(w, merges):
    syms = list(to_syms(w))
    for a, b in merges:
        i = 0
        while i < len(syms)-1:
            if syms[i] == a and syms[i+1] == b: syms[i:i+2] = [a+b]
            else: i += 1
    return syms
for w in ["orbiting", "spacecraft", "astrophysicist", "zzyzx", "reusability"]:
    seen = "in-vocab" if w in words else "NEVER SEEN"
    print(f"{w:15s} ({seen:10s}) -> {encode(w, merges)}")

fig, ax = plt.subplots(figsize=(5.4, 3.4))
ax.plot(sizes, lens, marker='o'); ax.set(xlabel='number of BPE merges', ylabel='avg
subwords per word',
        title='the vocabulary-size / sequence-length trade')
fig.tight_layout(); fig.savefig("figures/ch19/fig_l5_bpe.png", dpi=150)
print("figure saved: fig_l5_bpe.png")

```

Output:

```

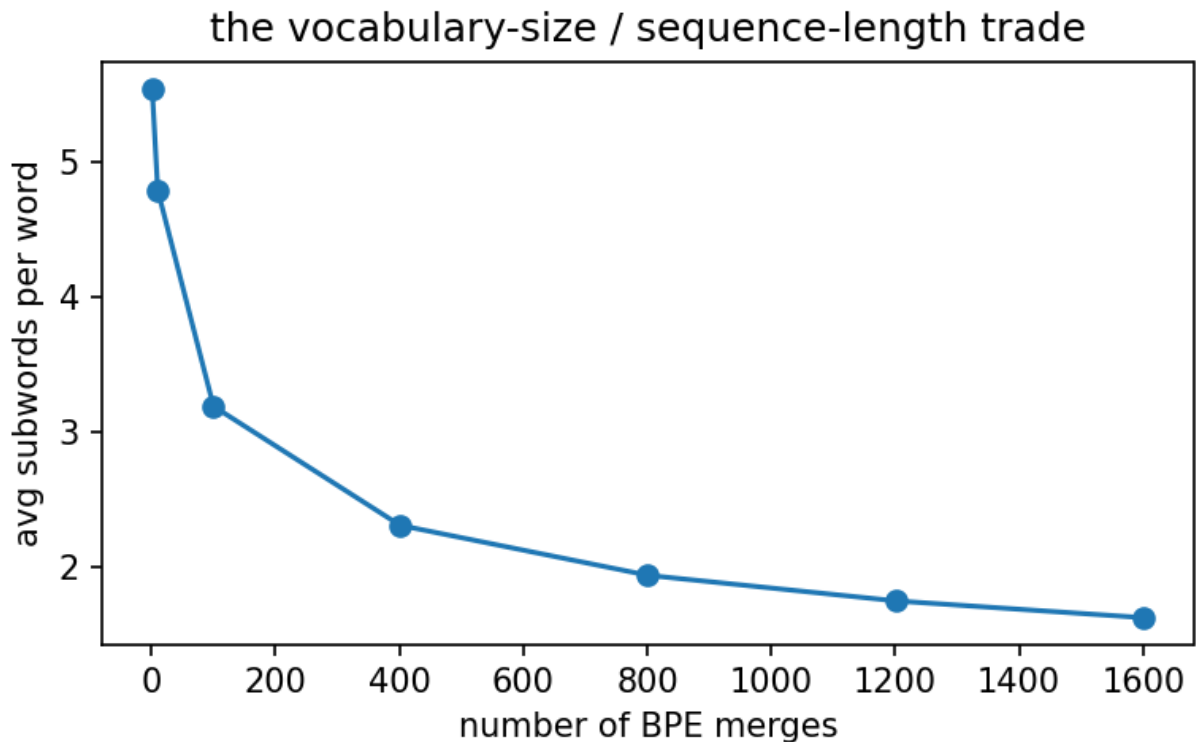
training words: 119,303 tokens, 12,140 types
after  1 merges: avg 5.54 subwords/word   latest merge: 'e'+ '</w>'
after 10 merges: avg 4.79 subwords/word   latest merge: 'th'+ 'e</w>'
after 100 merges: avg 3.19 subwords/word   latest merge: 'm'+ 'o'
after 401 merges: avg 2.31 subwords/word   latest merge: 'c'+ 'ur'

```

```

after 801 merges: avg 1.94 subwords/word   latest merge: 'ar'+'e'
after 1201 merges: avg 1.75 subwords/word  latest merge: 'b'+'it</w>'
after 1601 merges: avg 1.62 subwords/word  latest merge: 'vo'+'y'
orbiting      (in-vocab ) -> ['orbiting</w>']
spacecraft    (in-vocab ) -> ['spacecraft</w>']
astrophysicist (NEVER SEEN) -> ['astro', 'physici', 'st</w>']
zzyzx        (NEVER SEEN) -> ['z', 'z', 'y', 'z', 'x</w>']
reusability   (NEVER SEEN) -> ['re', 'us', 'ability</w>']
figure saved: fig_l5_bpe.png

```



### Listing 6 — An HMM part-of-speech tagger: Viterbi, verified against brute force

```

"""Listing 6: an HMM part-of-speech tagger -- Viterbi, verified against brute force."""
import numpy as np
from itertools import product
rng = np.random.default_rng(6)

TAGS = ["DET", "ADJ", "NOUN", "VERB", "ADV"]
WORDS = {"DET": ["the", "a", "this", "some"],
         "ADJ": ["red", "fast", "cold", "light", "back"],          # 'light', 'back', 'fast'
         "NOUN": ["rocket", "team", "goal", "ice", "light", "flies", "back", "fast"],
         "VERB": ["flies", "scores", "burns", "melts", "back", "ice"], #
         "ADV": ["quickly", "slowly", "today", "fast"]}
are AMBIGUOUS:
# they appear
under several tags
TRUE_T = {"DET": {"ADJ": .4, "NOUN": .6}, "ADJ": {"ADJ": .15, "NOUN": .85},
          "NOUN": {"VERB": .8, "NOUN": .2}, "VERB": {"DET": .5, "ADV": .3, "NOUN": .2},
          "ADV": {"DET": .6, "VERB": .4}}

def gen_sentence(maxlen=9):
    tag, out = "DET", []
    for _ in range(maxlen):

```

```

        w = WORDS[tag][rng.integers(len(WORDS[tag]))]
        out.append((w, tag))
        nxt = TRUE_T.get(tag); tags, ps = zip(*nxt.items())
        tag = tags[rng.choice(len(tags), p=np.array(ps))]
    return out
train = [gen_sentence() for _ in range(3000)]
test = [gen_sentence() for _ in range(300)]

# estimate HMM parameters by counting (add-0.1 smoothing)
T = len(TAGS); tag_i = {t: i for i, t in enumerate(TAGS)}
vocab = sorted({w for ws in WORDS.values() for w in ws}); word_i = {w: i for i, w in enumerate(vocab)}
A = np.full((T, T), .1); B = np.full((T, len(vocab)), .1); pi = np.full(T, .1)
for s in train:
    pi[tag_i[s[0][1]]] += 1
    for (w, t), (w2, t2) in zip(s, s[1:]): A[tag_i[t], tag_i[t2]] += 1
    for w, t in s: B[tag_i[t], word_i[w]] += 1
A /= A.sum(1, keepdims=True); B /= B.sum(1, keepdims=True); pi /= pi.sum()

def viterbi(ws):
    obs = [word_i[w] for w in ws]
    lp = np.log(pi) + np.log(B[:, obs[0]]); back = []
    for o in obs[1:]:
        scores = lp[:, None] + np.log(A) + np.log(B[:, o])[None]
        back.append(scores.argmax(0)); lp = scores.max(0)
    path = [int(lp.argmax())]
    for bk in back[::-1]: path.append(int(bk[path[-1]]));
    return [TAGS[i] for i in path[::-1]]

def brute(ws):
    obs = [word_i[w] for w in ws]; best, bp = -np.inf, None
    for path in product(range(T), repeat=len(ws)):
        s = np.log(pi[path[0]]) + np.log(B[path[0], obs[0]])
        for k in range(1, len(ws)):
            s += np.log(A[path[k-1], path[k]]) + np.log(B[path[k], obs[k]])
        if s > best: best, bp = s, path
    return [TAGS[i] for i in bp]

# exactness check on short sentences: Viterbi == exhaustive search
ok = all(viterbi([w for w, _ in s[:5]]) == brute([w for w, _ in s[:5]]) for s in test[:25])
print(f"Viterbi == brute-force argmax on 25 sentences (5^5 = 3125 paths each): {ok}")

def greedy(ws):
    obs = [word_i[w] for w in ws]
    path = [int((np.log(pi)+np.log(B[:, obs[0]])).argmax())]
    for o in obs[1:]:
        path.append(int((np.log(A[path[-1]]+np.log(B[:, o])).argmax()))
    return [TAGS[i] for i in path]

vt = gr = base = n = 0
mfreq = B.argmax(0)
# most-frequent-tag baseline per word
for s in test:
    ws, ts = zip(*s)
    vt += sum(a == b for a, b in zip(viterbi(list(ws)), ts))
    gr += sum(a == b for a, b in zip(greedy(list(ws)), ts))
    base += sum(TAGS[mfreq[word_i[w]]] == t for w, t in s)
    n += len(s)
print(f"most-frequent-tag baseline: {base/n:.3f}")
print(f"greedy left-to-right: {gr/n:.3f}")
print(f"Viterbi: {vt/n:.3f}")
s = ["the", "light", "flies", "quickly"]
print(f"\n{' '.join(s)}' -> greedy {greedy(s)}")
print(f"\n{' '.join(s)}' -> viterbi {viterbi(s)} (context disambiguates 'light' and 'flies')")

```



Output:

```
Viterbi == brute-force argmax on 25 sentences (5^5 = 3125 paths each): True
most-frequent-tag baseline: 0.746
greedy left-to-right:      0.807
Viterbi:                   0.956

'the light flies quickly' -> greedy ['DET', 'ADJ', 'NOUN', 'VERB']
'the light flies quickly' -> viterbi ['DET', 'NOUN', 'VERB', 'ADV'] (context
disambiguates 'light' and 'flies')
```

## Listing 7 — NER as BIO sequence labeling: a structured perceptron with Viterbi

```
"""Listing 7: NER as BIO sequence labeling -- a structured perceptron with Viterbi
decoding."""
import numpy as np
rng = np.random.default_rng(7)

PER = ["alice", "bob", "carol", "dave", "erin"]
ORG = ["nasa", "esa", "spacex", "boeing", "mit"]
LOC = ["houston", "moscow", "paris", "orlando", "tokyo"]
FILL = ["the", "launch", "was", "delayed", "by", "team", "from", "visited", "met", "with", "at", "in", "announced", "to"]
LABELS = ["O", "B-PER", "I-PER", "B-ORG", "I-ORG", "B-LOC", "I-LOC"]
L = len(LABELS); lab_i = {l: i for i, l in enumerate(LABELS)}

def gen():
    toks, tags = [], []
    for _ in range(rng.integers(6, 12)):
        r = rng.random()
        if r < .15:
            p = PER[rng.integers(5)]; toks += [p]; tags += ["B-PER"]
            if rng.random() < .4: toks += [PER[rng.integers(5)]]; tags += ["I-PER"]
        # double names
        elif r < .28:
            toks += [ORG[rng.integers(5)]]; tags += ["B-ORG"]
        elif r < .38:
            toks += [LOC[rng.integers(5)]]; tags += ["B-LOC"]
        else:
            toks += [FILL[rng.integers(len(FILL))]]; tags += ["O"]
    return toks, tags

train = [gen() for _ in range(1500)]; test = [gen() for _ in range(300)]
# make it hard: entity words sometimes appear lowercase as fillers ('paris' the person
vs city etc.)
# feature template: current word, previous word, next word, word-is-known-entity-list
feats = {}
def fid(f):
    if f not in feats: feats[f] = len(feats)
    return feats[f]
def phi(toks, i):
    prev = toks[i-1] if i else "<s>"; nxt = toks[i+1] if i < len(toks)-1 else "</s>"
    return [fid(("w", toks[i])), fid(("p", prev)), fid(("n", nxt))]

# structured perceptron: score = emission(features) + transition(prev_tag, tag);
Viterbi decode
for s, _ in train:
    for i in range(len(s)): phi(s, i)
W = np.zeros((L, len(feats))); Trans = np.zeros((L, L))

def decode(toks):
    F = [phi(toks, i) for i in range(len(toks))]
    em = np.array([W[l, f].sum() for l in range(L)] for f in F) # (n, L)
```

```

lp = em[0].copy(); back = []
for t in range(1, len(toks)):
    sc = lp[:, None] + Trans + em[t][None]
    back.append(sc.argmax(0)); lp = sc.max(0)
path = [int(lp.argmax())]
for bk in back[::-1]: path.append(int(bk[path[-1]]))
return path[::-1]

for epoch in range(5):
    err = 0
    for toks, tags in train:
        gold = [lab_i[t] for t in tags]
        pred = decode(toks)
        if pred != gold:
            err += 1
            for i in range(len(toks)):
                if pred[i] != gold[i]:
                    for f in phi(toks, i): W[gold[i], f] += 1; W[pred[i], f] -= 1
            for i in range(1, len(toks)):
                Trans[gold[i-1], gold[i]] += 1; Trans[pred[i-1], pred[i]] -= 1
    print(f"epoch {epoch+1}: sentence errors {err}/{len(train)}")

def spans(tags):
    out, cur = set(), None
    for i, t in enumerate(tags + ["0"]):
        if t.startswith("B-"):
            if cur: out.add(cur)
            cur = (t[2:], i, i)
        elif t.startswith("I-") and cur and cur[0] == t[2:]: cur = (cur[0], cur[1], i)
        else:
            if cur: out.add(cur)
            cur = None
    return out
tp = fp = fn = 0
for toks, tags in test:
    g = spans(tags); p = spans([LABELS[i] for i in decode(toks)])
    tp += len(g & p); fp += len(p - g); fn += len(g - p)
prec, rec = tp/(tp+fp), tp/(tp+fn)
print(f"span-level: precision {prec:.3f} recall {rec:.3f} F1 {2*prec*rec/(prec+rec):.3f}")
print("note: span F1 gives NO credit for half-found entities -- the standard (and brutal) NER metric")
toks = "alice bob met with spacex at houston today".split()
print(f"\n{' '.join(toks)}\n-> {[LABELS[i] for i in decode(toks)]}")

```

Output:

```

epoch 1: sentence errors 300/1500
epoch 2: sentence errors 217/1500
epoch 3: sentence errors 200/1500
epoch 4: sentence errors 204/1500
epoch 5: sentence errors 198/1500
span-level: precision 0.945 recall 0.927 F1 0.936
note: span F1 gives NO credit for half-found entities -- the standard (and brutal) NER metric

'alice bob met with spacex at houston today'
-> ['B-PER', 'I-PER', 'O', 'O', 'B-ORG', 'O', 'B-LOC', 'O']

```

## Listing 8 — Sentiment and the negation problem: where bag-of-words breaks

```

"""Listing 8: sentiment and the negation problem -- where bag-of-words breaks."""
import re, numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(8)

POS = ["good", "great", "excellent", "wonderful", "superb", "enjoyable", "brilliant"]
NEG = ["bad", "awful", "terrible", "boring", "poor", "dreadful", "weak"]
FILL = [
    "the", "movie", "was", "and", "plot", "acting", "i", "thought", "it", "really", "film", "overall", "script"]

def gen(n):
    """sentences with sentiment words, ~40% of them NEGATED ('not good' etc.)"""
    docs, ys = [], []
    for _ in range(n):
        y = rng.integers(2)
        words = [FILL[rng.integers(len(FILL))] for _ in range(rng.integers(5, 9))]
        for _ in range(rng.integers(1, 3)):
            pol = y if rng.random() > .4 else 1-y          # 40%: opposite-polarity
            word...
            w = (POS if pol else NEG)[rng.integers(7)]
            if pol != y: w = "not " + w                    # ...but NEGATED, so
            meaning matches y
            words.insert(rng.integers(len(words)), w)
            docs.append(" ".join(words)); ys.append(y)
    return docs, np.array(ys)
Xtr, ytr = gen(3000); Xte, yte = gen(1000)
print("example:", repr(Xtr[2]), "-> label", ytr[2])

def featurize(docs, ngram):
    vocab = {}
    rows = []
    for d in docs:
        toks = d.split()
        grams = toks if ngram == 1 else [" ".join(toks[i:i+2]) for i in
range(len(toks)-1)] + toks
        rows.append(grams)
    for g in (g for r in rows for g in r):
        if g not in vocab: vocab[g] = len(vocab)
    X = np.zeros((len(docs), len(vocab)))
    for i, r in enumerate(rows):
        for g in r:
            if g in vocab: X[i, vocab[g]] += 1
    return X, vocab

def mark_negation(d):
    """the classic hack: rewrite 'not good' -> 'not good_NEG'"""
    toks = d.split(); out = []
    i = 0
    while i < len(toks):
        if toks[i] == "not" and i+1 < len(toks):
            out.append(toks[i+1] + "_NEG"); i += 2
        else:
            out.append(toks[i]); i += 1
    return " ".join(out)

def run(tr_docs, te_docs, ngram, name):
    Xa, vocab = featurize(tr_docs, ngram)
    Xb = np.zeros((len(te_docs), len(vocab)))
    for i, d in enumerate(te_docs):
        toks = d.split()
        grams = toks if ngram == 1 else [" ".join(toks[i2:i2+2]) for i2 in
range(len(toks)-1)] + toks

```

```

    for g in grams:
        if g in vocab: Xb[i, vocab[g]] += 1
    w = np.zeros(Xa.shape[1]); b = 0.0
    for _ in range(200):
        p = 1/(1+np.exp(-np.clip(Xa@w+b, -30, 30)))
        w -= .5*Xa.T@(p-ytr)/len(ytr); b -= .5*(p-ytr).mean()
    acc = (((Xb@w+b) > 0) == yte).mean()
    print(f"{name:38s}: test acc {acc:.3f}")
    return acc
a1 = run(Xtr, Xte, 1, "unigram bag-of-words")
a2 = run(Xtr, Xte, 2, "unigrams + bigrams")
a3 = run([mark_negation(d) for d in Xtr], [mark_negation(d) for d in Xte], 1,
"unigrams + negation marking (_NEG)")

fig, ax = plt.subplots(figsize=(5.6, 3))
ax.bar(["unigrams", "uni+bigrams", "negation-marked"], [a1, a2, a3])
ax.set_ylim(.5, 1); ax.set_ylabel('test accuracy');
ax.set_title('the negation problem, three treatments')
fig.tight_layout(); fig.savefig("figures/ch19/fig_l8_negation.png", dpi=150)
print("figure saved: fig_l8_negation.png")

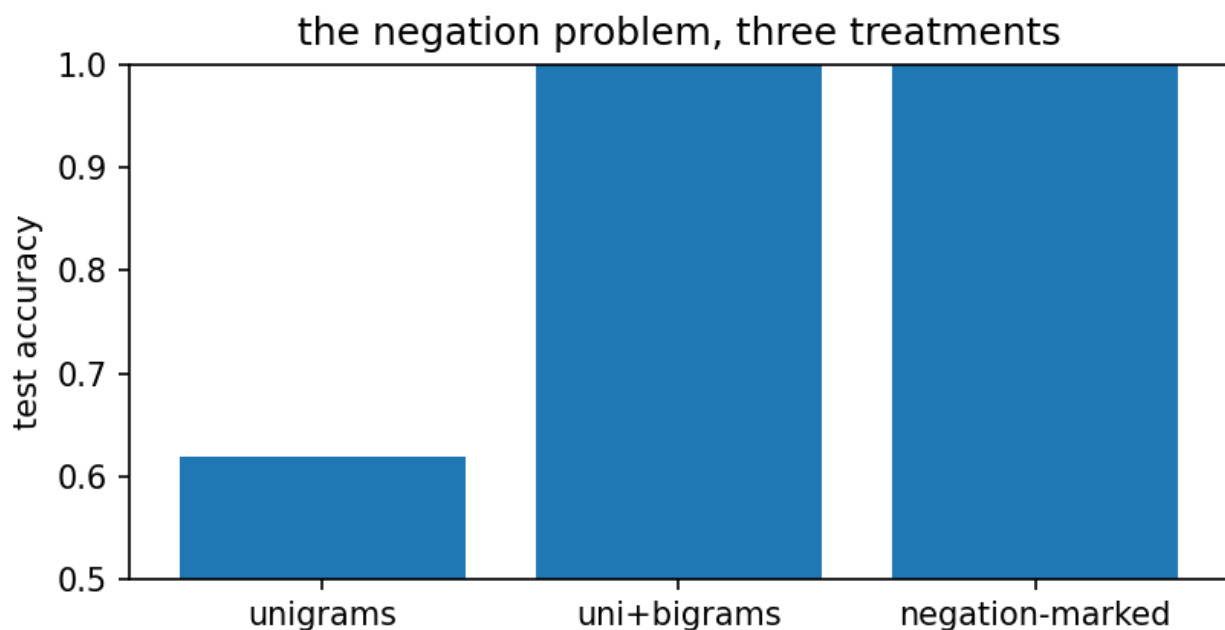
```

Output:

```

example: 'and acting poor thought acting thought' -> label 0
unigram bag-of-words           : test acc 0.619
unigrams + bigrams             : test acc 1.000
unigrams + negation marking (_NEG) : test acc 1.000
figure saved: fig_l8_negation.png

```



## Interview questions and answers

### Q1. State Zipf's law and two engineering consequences.

Word frequency is inversely proportional to rank:  $f(r) \sim r^{-1}$  — Listing 1 measures slope -0.97 on 20 Newsgroups. Consequence 1, vocabulary design: top-100 types cover 42% of tokens while 42% of TYPES are hapax legomena (seen once) — a fixed word vocabulary spends most slots on words with no training signal, motivating subword tokenization. Consequence 2, sparsity: n-gram counts thin out exponentially with n (most grammatical trigrams are unseen in any finite corpus), making smoothing the central problem of count-based LMs. Bonus: the fat tail is also why frequent-word subsampling helps word2vec — 'the' contributes millions of pairs and almost no signal.

### Q2. Stemming vs lemmatization — mechanism, failure modes, and when either still matters.

Stemming: rule-based suffix stripping (Porter), fast, no dictionary; produces non-words and errs both ways — Listing 1's toy rules: overstemming ('news' -> 'new', merging distinct meanings), non-words ('flies' -> 'fli'), understemming ('university' untouched). Lemmatization: dictionary + POS lookup to the canonical form ('better' -> 'good'); accurate, slower, resource-dependent. Both exist to fight sparsity by merging inflections. Today: subword tokenization made them obsolete for neural models (morphemes are learned), but they persist in retrieval/search indexing, matching pipelines, and small-data classical baselines where vocabulary reduction still pays.

### Q3. Write the TF-IDF formula and justify each factor.

$\text{weight}(t, d) = \text{tf}(t, d) \times \log(N/\text{df}(t))$ , rows L2-normalized. tf: how much the doc is about t. idf: log of inverse document frequency — a word in every document ( $\text{df}=N$ ) gets  $\log 1 = 0$  weight regardless of count ('the' vanishes without a stopwords list); a word in one document gets  $\log N$ . The log tempers the range; +1 smoothing avoids  $\text{df}=0$ ; L2 normalization removes document-length bias and makes dot products cosine similarities. Measured value (Listing 2): 0.919 -> 0.943 over raw counts. Worth volunteering: BM25 is TF-IDF's retrieval descendant with saturating tf and length normalization — still the lexical-search standard (Chapter 24).

**Q4. Why does add-one smoothing wreck n-gram models? What actually works?**

Laplace adds 1 to ALL  $V$  continuations of every context. With  $V=6,350$  and typical context counts of tens, the fake mass ( $V \cdot k = 6,350$ ) dwarfs the real observations — nearly all probability goes to never-seen continuations. Measured (Listing 3): trigram+Laplace perplexity 3,891 vs 438 for a plain UNIGRAM model — smoothing made the stronger model 9x worse;  $k=0.01$  still 1,653. What works: interpolation across orders ( $0.5 \cdot \text{tri} + 0.3 \cdot \text{bi} + 0.2 \cdot \text{uni} \rightarrow 290$ ), Katz backoff, and Kneser-Ney — whose extra idea is continuation probability:  $P_{\text{cont}}(w) \sim \text{number of distinct contexts } w \text{ completes}$ , so 'francisco' (frequent but only after 'san') gets low unigram backoff weight. KN was state of the art until neural LMs.

**Q5. Define perplexity, interpret it, and give its comparability caveat.**

PPL =  $\exp(\text{mean negative log-likelihood per token})$  — the effective branching factor: PPL 290 means the model is as uncertain as a uniform choice among 290 continuations at each step. Lower is better; a uniform model over  $V$  scores  $V$ . Caveats: comparable ONLY at identical vocabulary and tokenization (different policies or subword schemes change the token count and the per-token difficulty — a byte-level model's PPL is not a word-level model's); sensitive to the test domain; and it measures FIT, not usefulness (Chapter 20's models optimize it, Chapter 21 explains why that's not enough). Per-token log base matters too: report nats or bits consistently.

**Q6. CBOW vs skip-gram, and why negative sampling exists. Derive the SGNS gradient.**

CBOW predicts the center from averaged context (fast, smooths rare words away); skip-gram predicts each context from the center (slower, better rare-word vectors — each occurrence generates  $2W$  training pairs). Honest objective: softmax over  $V$  per step —  $O(V)$  per update, unaffordable. Negative sampling replaces it: maximize  $\log \text{sig}(v_c \cdot u_o) + \sum \text{over } k \text{ sampled negatives of } \log \text{sig}(-v_c \cdot u_n)$ , negatives from  $\text{unigram}^{(3/4)}$ . Gradients (Listing 4's lines):  $d/dv_c = (\text{sig}(v_c \cdot u_o) - 1) \cdot u_o + \sum \text{sig}(v_c \cdot u_n) \cdot u_n$ ; symmetric for  $u$ 's. It is logistic discrimination of true from noise pairs (noise-contrastive flavor), and Levy-Goldberg showed SGNS implicitly factorizes the PMI matrix shifted by  $\log k$ .

**Q7. Your word embeddings show cosine similarity 0.99 between everything. Diagnose.**

Two candidates, both in Listing 4. (1) Collapse from undertraining or a bug: vectors barely moved from init plus a shared drift direction (negative-sampling updates push everything similarly early on) — the listing's first run used one context per center and produced universal 0.998 cosines; the fix was all 2W contexts per center, more epochs, lr decay. (2) Anisotropy, the benign version: healthy embeddings still share a dominant common direction — raw mean cosine 0.59 even in the GOOD run; subtract the mean vector (all-but-the-top) before measuring: mean cosine drops to 0.10 and neighbors become semantic. Diagnostic order: check mean pairwise cosine AFTER centering; if still  $\sim 1$ , it's training, not geometry.

**Q8. GloVe vs word2vec vs FastText in one pass.**

word2vec (SGNS): local windows, online SGD on discrimination; implicitly factorizes shifted PMI. GloVe: explicit global objective — weighted least squares fitting  $w_i \cdot w_j + \text{biases}$  to  $\log X_{ij}$  (co-occurrence counts); same quality class, one pass over counts instead of the corpus. FastText: word vector = sum of character-n-gram vectors (3-6 chars) — handles OOV (compose a vector for unseen words from their n-grams), morphology-aware, and better for rare words and morphologically-rich languages; costs memory. Shared limitation: STATIC — one vector per type, senses collide ('bank'), no context sensitivity — the gap contextual embeddings (ELMo, BERT; Chapter 20) close.

**Q9. Walk through the BPE training loop and explain the vocabulary-size trade-off.**

Represent words as character sequences + end-of-word marker; count all adjacent symbol pairs (weighted by word frequency); merge the most frequent pair everywhere; repeat for a fixed number of merges — that merge list IS the tokenizer (apply merges in learned order at inference). Listing 5: first merges are 'e+', 'th+e'; 2,000 merges take avg subwords/word 5.5  $\rightarrow$  1.6. The trade (the listing's curve): bigger vocab  $\rightarrow$  shorter sequences (cheaper attention, more parallel signal) but bigger embedding/softmax matrices and rarer per-token updates; smaller vocab  $\rightarrow$  the reverse. Production LLMs sit at 32k-256k. OOV is dead: unseen words decompose to known subwords, worst case characters ('zzyzx'  $\rightarrow$  z z y z x).

**Q10. BPE vs WordPiece vs Unigram/SentencePiece — what actually differs?**

BPE: merge by raw pair FREQUENCY, bottom-up, deterministic greedy segmentation. WordPiece (BERT): merge by LIKELIHOOD gain — pick the pair that most increases a unigram LM's corpus likelihood (frequency normalized by the parts' frequencies); ## continuation prefix. Unigram LM: TOP-DOWN — start with a large candidate set, iteratively prune tokens whose removal least hurts likelihood; yields a probabilistic tokenizer enabling subword regularization (sample segmentations as augmentation). SentencePiece: the toolkit — operates on raw text with spaces as symbols (no whitespace pre-tokenization, language-agnostic; implements BPE and Unigram). Byte-level BPE (GPT-2+): base alphabet = 256 bytes, so ANY string tokenizes. Interview hook: tokenization explains LLMs' character-counting and arithmetic fragility — the model never sees letters.

**Q11. Define the HMM tagging model and derive Viterbi's recurrence. What guarantees exactness?**

Generative model:  $p(\text{tags}, \text{words}) = \prod A[t_{(i-1)}, t_i] \times B[t_i, w_i]$  (transitions  $\times$  emissions), parameters by smoothed counting. Decoding wants  $\text{argmax over } T^n$  paths. Viterbi:  $\text{delta}_i(t) = \text{best score of any path ending in tag } t \text{ at position } i = \max_{t'} \text{ of } \text{delta}_{(i-1)}(t') + \log A[t', t] + \log B[t, w_i]$ ; keep backpointers, trace back from the best final tag.  $O(n \cdot T^2)$ . Exactness holds because the score DECOMPOSES over adjacent pairs (first-order Markov): the best path ending in  $t$  at  $i$  must extend some best path at  $i-1$  — optimal substructure. Listing 6 verifies it against brute-force enumeration of all  $5^5$  paths: identical. Longer-range interactions break the guarantee (higher-order HMMs expand the state space instead).

**Q12. Why does Viterbi beat greedy decoding? Give the measured gap and a concrete sentence.**

Greedy commits to the best tag at each position given only the past; joint decoding lets LATER evidence veto earlier choices. Listing 6, with genuinely ambiguous words ('light' ADJ/NOUN, 'flies' NOUN/VERB): baseline (most-frequent tag) 0.746, greedy 0.807, Viterbi 0.956. The garden path: 'the light flies quickly' — greedy takes 'light' as ADJ (locally likely after DET), then 'flies' as NOUN, then hits 'quickly' with no good continuation; Viterbi returns DET NOUN VERB ADV because the path score including the adverb favors reinterpreting the front. Same phenomenon at LLM scale: greedy vs beam search (Chapter 22).



**Q13. Explain BIO tagging and why NER is evaluated with span F1. What does the metric punish?**

BIO: B-TYPE begins an entity, I-TYPE continues it, O is outside — turning span extraction into per-token labeling while preserving boundaries (two adjacent PER entities: B-PER I-PER vs B-PER B-PER distinguishes 'alice bob' as one vs two people — Listing 7's example tags it correctly). Span F1: a predicted entity counts only if type AND boundaries match exactly; precision/recall over whole spans. It punishes partial credit deliberately — finding 'York' inside 'New York' is worth zero, because downstream consumers (linking, extraction) need exact spans. Listing 7's structured perceptron: 0.936 span F1. Also name scheme variants (BILOU/BIOES) and constraint: I-X cannot follow O — decode with transition constraints or fix in post.

**Q14. Describe the structured perceptron update and its relation to CRFs.**

Score a tag sequence as sum of emission features + transition weights; decode with Viterbi; if the predicted path differs from gold, add features of the GOLD path and subtract features of the PREDICTED path (both emissions and transitions) — Listing 7's inner loop, converging to 0.936 F1 in 5 epochs. Relation to CRF: identical model form (linear scores over the same feature templates, global decoding); the CRF replaces the perceptron's argmax update with gradient of the globally-normalized log-likelihood, requiring forward-backward to compute expectations instead of just Viterbi. Perceptron: simpler, no probabilities, surprisingly competitive; CRF: calibrated distributions, marginals per token, the standard until BiLSTM-CRF and then transformer fine-tuning absorbed it.

**Q15. When does TF-IDF + logistic regression beat a fine-tuned transformer? Be concrete.**

(1) Small labeled data with strong LEXICAL signal: topic routing, spam, language ID — Listing 2 hits 0.943 with 300 gradient steps on a CPU. (2) Latency/cost floors: sparse dot products serve at microseconds; no GPU. (3) Interpretability/audit requirements: Listing 2's weights ARE the explanation (nasa/orbit vs hockey/season) — regulators can read them. (4) Distribution drift you must debug fast: retrain in seconds. (5) Very long documents where truncation hurts transformers but counts don't care. The transformer wins when labels hinge on composition, context, or world knowledge (Listing 8's negation is the minimal example), or when transfer from pretraining outweighs the label shortage. Professional answer: always BUILD the linear baseline first — it sets the bar and costs nothing.

**Q16. The negation problem: show the failure and rank the classical fixes.**

Bag-of-words scores a document by summed word evidence; 'not good' contributes the SAME 'good' as 'very good'. Listing 8, with 40% negated polarity words: unigrams 0.619 (errors are exactly the negated docs). Fixes measured at 1.000: (a) bigrams — 'not good' becomes an atomic feature; cost: squared vocabulary, sparsity; (b) negation marking — rewrite tokens after a negator as word\_NEG until a clause break (the pre-neural sentiment standard; cheap, targeted). Ranked: negation marking when negation is THE problem (tiny vocab growth); bigrams when local composition generally matters; syntactic scope handling for 'not only... but' subtleties; neural attention when composition is pervasive — each fix is a hand-crafted shadow of what self-attention learns.

**Q17. What's the OOV problem in each era: word vocabularies, static embeddings, subword LLMs?**

Word-count era: unseen test words are DROPPED (Listing 2 builds vocab from train only) or mapped to (Listing 3, words with count < 2) — information destroyed, perplexity/accuracy costs. Static embeddings: no vector for unseen words — FastText composes one from character n-grams; otherwise again. Subword era: OOV is structurally dead — BPE decomposes anything ('astrophysicist' -> astro+physici+st, Listing 5), worst case bytes. What remains: RARE-token fragility (few updates per subword during training), token-boundary artifacts (leading-space tokens, digit splitting), and domain terms shattered into many pieces — 'unseen' became 'expensive and poorly learned' rather than 'impossible'.

**Q18. Sketch the dependency-parsing landscape in interview depth.**

Dependency parse: directed tree of head->modifier arcs with labels (nsubj, dobj). Transition-based: parse as a sequence of shift/reduce actions over a stack+buffer, greedy or beam,  $O(n)$ , features/BiLSTM/BERT over the configuration — fast, local errors propagate. Graph-based: score every possible arc, find the maximum spanning tree (Chu-Liu-Edmonds), exact global argmax,  $O(n^2)$  — accuracy edge on long-distance attachments. Biaffine attention (Dozat-Manning) is the modern standard scorer. Uses: feature extraction for IE/QA pre-LLM; today mostly supplanted end-to-end, but parsing questions persist as tests of structured-prediction fluency — and Viterbi/MST/CKY are the same dynamic-programming family.

**Q19. How would you build language identification for 100 languages with 10ms latency?**

Character n-gram features (1-4 grams, hashed to a fixed dimension) + multinomial logistic regression or linear SVM — the fastText-style recipe: languages differ most in character statistics, words are unnecessary, and hashing kills the vocabulary problem. Training: balanced corpora per language, augment with romanization/code-switching examples, and short-text augmentation (language ID is easy at paragraph length, hard at 3 words — train on truncations). Evaluate per-language recall (macro, not micro — rare languages matter), confusion clusters (Bokmal/Nynorsk, Serbian/Croatian), and a rejection threshold for out-of-set inputs. Serving: a single sparse dot product — microseconds. A transformer here is engineering malpractice; this is Q15's judgment call in its purest form.

**Q20. Compare HMM and CRF as models of tagging — the generative/discriminative distinction in this setting.**

HMM: generative — models  $p(\text{tags, words})$  with emission  $p(\text{word}|\text{tag})$ ; parameters by counting; assumes words independent given tags, so overlapping features (capitalization AND suffix AND word identity) double-count evidence and can't be combined honestly. CRF: discriminative — models  $p(\text{tags}|\text{words})$  directly, globally normalized over tag sequences; arbitrary overlapping features of the ENTIRE observation at every position enter one linear score without independence claims. Practical consequences: HMM trains from counts in one pass (Listing 6) and can generate; CRF needs iterative optimization (forward-backward for the partition function) but dominates accuracy with rich features. Chapter 4's generative-vs-discriminative table, instantiated in sequences.

**Q21. Estimate memory and compute for TF-IDF logistic regression vs a 110M-parameter transformer on 10k docs.**

TF-IDF+LR: vocab 20k  $\rightarrow$  weight vector 20k floats = 80 KB; features sparse ( $\sim 100$  non-zeros/doc)  $\rightarrow$  training 300 epochs  $\times$  10k docs  $\times$  100 mults  $\sim 3e8$  FLOPs — sub-second on a CPU; inference  $\sim 100$  FLOPs/doc. Transformer (BERT-base): 110M params = 440 MB fp32; fine-tuning 3 epochs  $\times$  10k docs  $\times$   $\sim 2 \times 10^{12}$  GFLOPs-ish per doc — GPU-hours; inference tens of GFLOPs/doc,  $\sim 10$ ms on GPU. Six-plus orders of magnitude in both dimensions. The transformer buys composition, context, and transfer; whether the task needs them is Q15. Doing this arithmetic aloud, roughly and quickly, is exactly what systems-minded interviewers reward.

**Q22. What is PMI, and how does it connect counts to embeddings?**

Pointwise mutual information:  $\text{PMI}(w, c) = \log p(w, c) / (p(w)p(c))$  — how much more often  $w$  and  $c$  co-occur than independence predicts; positive PMI (clip at 0) fixes the -inf of unseen pairs. Classical use: PPMI-weighted co-occurrence matrix + SVD = strong static embeddings before word2vec. The connection (Levy & Goldberg 2014): skip-gram with  $k$  negatives implicitly factorizes the matrix  $M = \text{PMI}(w, c) - \log k$  — SGNS is count-based matrix factorization in SGD clothing, which explains why GloVe (explicit count fitting) and word2vec land in the same quality class. Interview one-liner: 'neural' word embeddings are a factorization of the same co-occurrence statistics the classical methods counted.

**Q23. Design the text pipeline for a support-ticket router (40 classes, 50k historical tickets, new products monthly).**

Baseline first (Q15): TF-IDF (word 1-2 grams + char 3-5 grams for typos/product codes) + linear model, class-weighted for imbalance; ship it, measure per-class F1. Known traps: product names are entities — char n-grams and a gazetteer feature handle new SKUs; templates/signatures inflate lexical signal — strip boilerplate (Listing 2's remove= analog); temporal split for evaluation (new products monthly = distribution drift; random splits lie, Chapter 4). Upgrade path: fine-tuned small transformer when composition errors dominate the confusion matrix; hybrid routing (linear for high-confidence, model for the tail). Monitoring: per-class drift on incoming vocabulary (OOV-rate spike = new product), monthly re-train, human-in-the-loop for low-confidence. The design answer is the ORDER: baseline, error analysis, then capacity.

**Q24. Why does word2vec use two embedding matrices, and what do people do with them afterward?**

Input matrix  $W_i$  (center vectors) and output matrix  $W_o$  (context vectors): a word's role as PREDICTOR differs from its role as PREDICTED — with one shared matrix,  $\text{sig}(v_w \cdot v_w)$  pushes every word to co-occur with itself, distorting geometry (and the true objective is asymmetric: subsampled centers, noise-sampled contexts).

Afterward: use  $W_i$  alone (standard, Listing 4), average  $W_i + W_o$  (GloVe's habit, small consistent gains), or keep both for directional tasks. The same input/output split reappears in LLMs as the embedding vs unembedding matrices — and weight TYING them (one matrix for both) became standard there for parameter savings, with the asymmetry absorbed by the network in between.

**Q25. A colleague reports 97% accuracy on sentiment with a model trained and evaluated on reviews that include star ratings in the text. Critique.**

Leakage (Chapter 4, textual edition): the label is printed in the features — '5 stars' or the digit itself; the model reads the rating, not the sentiment. Checks: top LR weights (Listing 2's technique — if '5', 'stars', 'refund' dominate, it's leakage); accuracy after regex-stripping ratings/URLs/metadata; per-source breakdown. Related textual leaks: dataset artifacts (review length, platform boilerplate), duplicate reviews across train/test (dedupe by near-match, not exact string), and temporal leakage (same product's reviews split across sets share phrasing). Honest protocol: strip metadata, dedupe near-duplicates, split by product AND time, then re-measure — the 97% will find its real level.

**Q26. What did ELMo change relative to word2vec, one step before transformers?**

Contextual embeddings: a word's vector is the hidden state of a deep bidirectional LSTM language model AT THAT OCCURRENCE — 'bank' by the river and 'bank' with a vault finally get different vectors, killing the static-embedding sense collision (Q8). Mechanically: pretrain forward and backward LSTMs on next/previous-token prediction, represent each token as a learned task-specific weighted sum of layer states, feed into task models as features. It delivered large gains across tasks in 2018 and established the recipe — pretrain a big LM on unlabeled text, transfer to tasks — that BERT (masked, transformer, fine-tuned end-to-end) industrialized one year later (Chapter 20). Bridge fact: ELMo's layers specialize (lower = syntax, higher = semantics), foreshadowing probing results in transformers.

**Q27. Your BPE tokenizer was trained on English web text; the product now serves Hindi and code. What breaks and what do you do?**

Breaks: Hindi words shatter into bytes/characters (the merges encode English statistics) — sequences 3-8x longer, so effective context shrinks, latency and cost multiply, and rare-piece embeddings are undertrained -> quality drops; code suffers similarly (identifiers split unnaturally, whitespace semantics lost unless the tokenizer preserved it). Diagnose: tokens-per-character by language; embedding coverage. Options, cheapest first: (1) accept and budget (short-term); (2) extend vocabulary — train added merges on new-domain text, initialize new embeddings from subword averages, continue pretraining briefly; (3) retrain tokenizer + model (the real fix, expensive); (4) route to a multilingual model (SentencePiece trained on balanced multilingual data). Prevention: train tokenizers on the deployment mixture — tokenizer choices are locked in earlier than any other model decision.

**Q28. This chapter's measured claims, end to end.**

Zipf slope -0.97; 42% hapax; 19 stopwords = 24% of tokens; stemming's three failure modes on camera (Listing 1). TF-IDF +2.4 points over counts; learned weights recover topic lexicons; train-only vocabulary = honest OOV (Listing 2). Laplace  $k=1$  makes trigrams 9x WORSE than unigrams (3,891 vs 438); interpolation wins at 290 (Listing 3). SGNS from scratch: nasa->ames/jsc, cos(shuttle,goalie) -0.80; collapsed-run lesson + anisotropy centering 0.59->0.10 (Listing 4). BPE: 5.5 -> 1.6 subwords/word over 2,000 merges; astrophysicist -> astro+physici+st; OOV dead (Listing 5). Viterbi == brute force on 3,125 paths; 0.746/0.807/0.956 baseline/greedy/Viterbi (Listing 6). Structured perceptron BIO NER, span F1 0.936 (Listing 7). Negation: unigrams 0.619, bigrams or \_NEG marking 1.000 (Listing 8).